

CortexAI Ø>Ýà&j

Autonomous Multi-Agent Architecture & Algorithm Specification Document

CortexAI is a full-stack, enterprise-grade AI intelligence system engineered to surpass traditional single-prompt chatbots. Utilizing an Adaptive Orchestrator, CortexAI assesses task complexity, decomposes requests into dependency graphs, executes specialized domain models in parallel, and synthesizes structured executive outputs with live metrics and interactive code artifacts.

1. Distributed Microservices Topology

The CortexAI backend is decoupled into specialized microservices coordinated through an intelligent central API Gateway:

- API Gateway (Port 8000): Central reverse proxy handling dual session authentication, cookie/bearer header resolution, request routing, and streaming SSE multiplexing.
- Auth Service (Port 8001): Firebase token verification, MongoDB user persistence, and Redis session store management.
- Chat Service (Port 8002): Conversation threads, message history persistence, and chat telemetry.
- Agent Service (Port 8003): Dynamic task decomposition, DAG orchestration, LangGraph state machine, RAG retrieval, PPT/PDF generation, and observability.
- Billing Service (Port 8004): Razorpay payment processing, subscription quotas, and real-time credit metering.

2. Comprehensive Algorithmic Inventory

Algorithm 1: Heuristic Fast-Path Query Classifier

Evaluates prompts against deterministic regexes and file attachments to route zero-overhead requests directly without calling an expensive LLM analyzer.

- File Check: PDF attachments route to pdfRag; Images route to imageAnalyzer.
- Greetings & Queries: Conversational greetings (hi, hello) route immediately to Chat Agent (Complexity: 1).
- Direct Actions: Matches 'create ppt', 'create pdf', 'write code' to specialized agents.

Algorithm 2: Dynamic Task Decomposition & Scientific Telemetry Scoring

Uses LLM-guided meta-planning to generate atomic subtasks with execution strategies (single, parallel, sequential, hybrid) and 1-10 scientific scores:

- Complexity Score: 1 = Trivial knowledge lookup, 10 = Deep multi-dimensional synthesis.
- Decomposability Score: 1 = Atomic query, 10 = Highly parallelizable sub-problems.
- Dependency Score: 1 = Fully independent subtasks, 10 = Strict sequential pipeline.
- Tool Requirement Score: 1 = Pure reasoning, 10 = Heavy live search / RAG necessity.
- Risk Score: 1 = High certainty, 10 = Ambiguous or multi-hypothesis inquiry.

Algorithm 3: Dynamic DAG Topological Batch Orchestrator

Iteratively resolves dependencies among subtasks, extracting ready batches where all parent dependencies are fulfilled, executing them concurrently using Promise.allSettled(), and passing accumulated context to downstream agents.

1. Initialize CompletedSet C = empty
2. While remaining subtasks > 0:
 - a. ReadyBatch B = { s in Remaining | s.dependencies subset of C }
 - b. If B is empty: B = [Remaining.shift()] // Prevent cycles
 - c. Execute all s in B concurrently via Promise.allSettled()
 - d. Accumulate outputs & metrics into Context Store
 - e. C = C union { s.id for s in B }
3. Execute Synthesizer Agent on all accumulated findings.

Algorithm 4: Distributed Real-Time Cancellation Lifecycle

Handles instant termination of active agent pipelines when the user clicks [Stop]. Uses Redis Pub/Sub channels (execution_events:<id>) and local memory caches to trigger ResearchCancelledError before and after every asynchronous agent invocation.

Algorithm 5: PDF RAG (Chunking, Vector Embeddings & Fallback Search)

Parses uploaded PDFs using pdf-parse, splits text with RecursiveCharacterTextSplitter (1000-char chunks, 200-char overlap), indexes into Qdrant vector database for k-NN similarity search (k=5), with an automatic tokenized keyword-frequency fallback when Qdrant is unavailable.

Algorithm 6: Search Normalization, Deduplication & Ranking

Cleans raw Tavily web search results, removes JSON noise and HTML tags, deduplicates URLs, limits snippets to 400 characters, extracts publisher domains, and formats clean markdown references.

Algorithm 7: Token Normalization & Multi-Model Cost Tracker

Normalizes inconsistent provider token metadata across Groq, Gemini, OpenRouter, and DeepSeek, computing exact USD costs per execution using calibrated pricing tables.

Algorithm 8: Executive Map-Reduce Multi-Agent Synthesizer

Consolidates multi-agent subtask outputs and verified web sources into a structured executive report with clean markdown tables, thematic sections, numbered takeaways, and download links.

Algorithm 9: Coding Intent Classification & Multi-File Generator

Classifies developer requests into CODE_GENERATION, REVIEW, EXPLANATION, or DEBUGGING. For generation, outputs strict JSON file trees rendered directly into the interactive Monaco Editor with live iframe preview.

Algorithm 10: Automated Presentation & PDF Rendering Engines

Programmatically builds styled 16:9 PowerPoint decks using pptxgenjs and styled PDF documents using pdfkit with automatic headers, zebra stripes, and page numbering.

Algorithm 11: Dual-Session Authentication & Forwarding

Inspects cookies, Authorization Bearer headers, and x-session-id headers, verifies Redis session tokens, and forwards authenticated x-user-id headers to downstream microservices.

3. Key API Endpoints & Routes

- POST /api/agent/chat: Primary endpoint for Adaptive Multi-Agent Orchestration.
- GET /api/agent/executions/:id/stream: Server-Sent Events (SSE) live progress & graph stream.
- POST /api/agent/executions/:id/cancel: Instant execution cancellation trigger.

- GET /api/agent/research/analytics: Aggregated scientific strategy analytics.
- POST /api/billing/create-order: Razorpay credit top-up order generation.

